FSF3827 Assignment 2

Anh Do
020416-2317
anhd@kth.se

November 14, 2025

Question 1

No. They never use the phrase "branch-and-bound," and they don't say who coined it. In their 2010 introduction, they state, "We did not initially think of the method as 'branch and bound'". They mention they aren't sure if the term was already in the literature. They describe their automatic method in terms of splitting the problem into sets of solutions by adding extra constraints, and using "upper bounds" from the linear-programming relaxation to decide what to explore or discard. The paper doesn't give us any explicit author for "branch-and-bound" other than mentioning, "Much later someone wrote a paper about 'shoulder branch and bound'".

The method selects a branching variable by finding the variable that is "furthest from an integer". After selecting a variable (like x_r), it "floors and ceils" it by creating two new problems: one with the new constraint that x_r equals the integer just below its value (e.g., $[x_r^0]$), and one where x_r equals the integer just above (e.g., $[x_r^0] + 1$).

It then finds the solution (the γ value) for both of these new problems and selects the one with the largest (maximize) objective value as the new best upper bound, or γ^1. The model then continues by taking this new problem, which now has a fixed integer value for x_r, and repeats the process by selecting the next variable that is furthest from an integer.

When an integer solution is found, it is only proven to be the optimum if its objective value is higher than the upper bounds of all other unexplored branches. The algorithm maintains a list of all potential branches and their γ values, always exploring the one with the highest value. The computation is complete only when an integer solution is found and its γ value is higher than any other γ value remaining on the list.

Question 2

Based on the survey paper, the main node selection (or search) strategies mentioned are Depth-First Search (DFS), Breadth-First Search (BrFS), Best-First Search (BFS), and Cyclic Best-First Search (CBFS). The order of exploration is important because it significantly affects computation time and memory requirements, finding a good incumbent solution early in the search allows the algorithm to prune more of the search tree, thus reducing the total number of nodes that must be explored to verify optimality.

The paper discusses branching strategies, which are categorized into binary and wide branching, and also mentions specific variable selection rules for integer programming, such as pseudocost branching, strong branching, and the "most fractional" rule. However, the paper states that the "most fractional" rule is not a good strategy, noting that it is generally no better than selecting a branching variable at random in terms of computational time or the number of subproblems explored.

Question 3

Based on the "Experiments in mixed-integer linear programming" paper, pseudo-costs are computed automatically during the branch-and-bound tree scan to measure the "importance" of each integer variable. When the algorithm branches at a node k on a non-integer variable y_b (which has a value $\overline{y}_b^k$ with integer part $[\overline{y}_b^k]$ and fractional part f_b^k), it creates a "down" node $(n+1)$ and an "up" node $(n+2)$.

The lower pseudo-cost (PCL_b) is calculated as the change in the objective function on the down branch divided by the fractional part:

$$PCL_b = \left| \frac{\overline{F}_{n+1} - \overline{F}_k}{f_b^k} \right|$$

The upper pseudo-cost (PCU_b) is the change on the up branch divided by one minus the fractional part, representing the objective's deterioration per unit of change:

$$PCU_b = \left| \frac{\overline{F}_{n+2} - \overline{F}_k}{1 - f_b^k} \right|$$

This pseudo-cost are calculated reactively.

These pseudo-costs are then used in two primary ways:

1. **To select the next branching node (via "estimations"):** An estimation $\overline{E}_k$ is calculated for each waiting node to forecast the best integer solution that can be expected from it. The node with the *best* (lowest) estimation is often chosen as the next node to explore. The formula is:

$$\overline{E}_k = \overline{F}_k + \sum_j \min(PCL_j \cdot f_j^k, PCU_j \cdot (1 - f_j^k))$$

   Here, $\overline{F}_k$ is the node's current objective value, and the sum represents the total *estimated penalty* to get to an integer solution, optimistically assuming each variable j will be branched on in its "cheaper" direction.

2. **To select the branching variable:** Once a node is selected, pseudo-costs are used to choose *which* fractional variable to branch on. The goal is to select the variable that causes the "greatest expected deterioration" of the objective, which can help prune branches faster. This is done by choosing the variable j that maximizes the minimum expected penalty:

$$\max_j \left(\min(PCL_j \cdot f_j^k, PCU_j \cdot (1 - f_j^k)) \right)$$

   This rule selects the variable whose *best-case* penalty (the min part) is still the *worst* (the max part) among all fractional variables, thus picking the variable that is "most expensive" to make integer.

Question 4

Strong Branching

Strong branching is a method for selecting a branching variable in an integer program. The core idea is to find the variable that will cause the "most change in the objective function". To do this, it runs a test: for a subset of candidate fractional variables, it temporarily solves the LP relaxation for both the "up" branch (adding $x_i \geq \lceil \overline{x}_i \rceil$) and the "down" branch (adding $x_i \leq \lfloor \overline{x}_i \rfloor$). After computing these new objective values for all candidates, it permanently selects the variable that gave the "best progress" (based on a score function) to be the *actual* branching variable for the current node. **Full strong branching** is when this test is performed for *all* fractional variables, which often leads to small search trees but is computationally expensive.

Reliability Branching

Reliability branching is a hybrid strategy that defaults to using fast **pseudocosts** but selectively uses the expensive **strong branching** calculation *only* when a variable's pseudocost score is "unreliable".

To be precise, the **pseudocost** (which you can think of as a better PCU or 'upward' score) is the **historical average** gain for a variable. The paper calls this average Ψ_i^+. This average is calculated using two other values:

- σ_i^+ is the **cumulative sum** of all *individual* upward gains (specifically, the ζ_i^+ values, which represent the same "per-branch" calculation as the old PCU from the previous problem) from *all previous encounters* where that variable was branched on.

- η_i^+ is the **count** of how many times that variable has been branched on in the "up" direction.

The **upward pseudocost** Ψ_i^+ is then the average: $\Psi_i^+ = \sigma_i^+ / \eta_i^+$.

Reliability branching **assesses "unreliability"** using these **counters** (η_i^- and η_i^+) and a user-defined "reliability parameter" called η_{rel}.

A variable i is officially considered **unreliable** if its historical count in *either* direction is less than this parameter. The exact check is:

$$\min\{\eta_i^-, \eta_i^+\} < \eta_{rel}$$

If a variable is deemed unreliable by this check, the algorithm will perform strong branching on it to get a high-quality objective gain value. This new result is then used to update the total sum (σ_i) and the count (η_i), making the average score (Ψ_i^+) more "reliable" for future decisions. If the variable's scores are already reliable (i.e., $\min\{\eta_i^-, \eta_i^+\} \geq \eta_{rel}$), the algorithm skips the expensive strong branching step and just trusts the stored pseudocost score.

Question 5

When solving the subproblems in strong branching, other useful information is gained.

First, variables can be fixed automatically. When a variable is tested, it might be found that setting it to one value (like 0) makes the problem impossible to solve, or infeasible. It is then proven that the variable must equal the other value (like 1) for any good solution. This rule can be added to the current problem without needing to branch, which makes the problem simpler and stronger.

Second, a branch can be pruned immediately. Both options (like 0 and 1) are tested. If the best integer solution found so far is 194, and the test of one option (like $x = 1$) shows its best possible LP value is only 190, that entire branch is known to be a dead end. That option can be discarded, and the search can be continued by forcing the other option ($x = 0$).